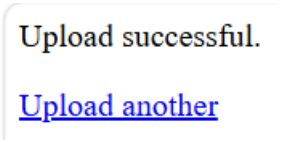
Screenshot of create_app_stack.sh completing, showing both http://:port/ URLs.

```
== Auto Scaling Groups (these launch the instances now, not run-instances) ==
Done. ALB DNS name: itmo-app-alb-1330632552.us-east-2.elb.amazonaws.com
Uploader: http://itmo-app-alb-1330632552.us-east-2.elb.amazonaws.com:8080/
Viewer:   http://itmo-app-alb-1330632552.us-east-2.elb.amazonaws.com:8081/
```

Screenshot of both apps working through the ALB (uploader upload confirmation, viewer showing content).





Screenshot of aws elbv2 describe-target-health showing both target groups healthy .

```
root@aa80cd592e15:/CloudClass# aws elbv2 describe-target-health --target-group-arn $UPLOADER_TG_ARN --region us-east-2
{
    "TargetHealthDescriptions": [
        {
            "Target": {
                "Id": "i-060c972ee372737c5",
                "Port": 3000
            },
            "HealthCheckPort": "3000",
            "TargetHealth": {
                "State": "healthy"
            },
            "AdministrativeOverride": {
                "State": "no_override",
                "Reason": "AdministrativeOverride.NoOverride",
                "Description": "No override is currently active on target"
            }
        }
    ]
}
root@aa80cd592e15:/CloudClass# aws elbv2 describe-target-health --target-group-arn $VIEWER_TG_ARN --region us-east-2
{
    "TargetHealthDescriptions": [
        {
            "Target": {
                "Id": "i-0de3f68ac381e8546",
                "Port": 3000
            },
            "HealthCheckPort": "3000",
            "TargetHealth": {
                "State": "healthy"
            },
            "AdministrativeOverride": {
                "State": "no_override",
                "Reason": "AdministrativeOverride.NoOverride",
                "Description": "No override is currently active on target"
            }
        }
    ]
}
```

The CSV poll log and console output from a chaos_test.py run that passed its SLO, for at least one tier.

```
root@aa80cd592e15:/CloudClass# python3 testing/chaos_test.py --tier uploader
Target: uploader tier at http://itmo-app-alb-1330632552.us-east-2.elb.amazonaws.com:8080/
Confirming the endpoint is healthy before we start...
Terminating i-060c972ee372737c5 via the Auto Scaling Group...
Waiting for the endpoint to go down...
Waiting for the endpoint to come back up...
Poll log written to chaos_test_uploader_34824.csv
Recovery time: 85.2s (SLO: 180s)
PASS: recovery time met the SLO.
```

```
root@aa80cd592e15:/CloudClass# cat chaos_test_uploader_34824.csv
elapsed_seconds,is_up
0.0,True
2.51,False
0.0,False
2.71,False
5.18,False
7.43,False
9.69,False
12.2,False
17.7,False
23.02,False
28.25,False
30.77,False
32.92,False
35.49,False
38.0,False
40.61,False
43.14,False
45.51,False
48.39,False
51.06,False
53.92,False
56.79,False
59.34,False
62.22,False
64.79,False
67.54,False
70.0,False
72.67,False
75.14,False
77.51,False
80.14,False
82.71,False
85.18,True
```

The CSV poll log and console output from a run that failed a tightened SLO.

```
root@aa80cd592e15:/CloudClass# python3 testing/chaos_test.py --tier uploader --slo-seconds 60
Target: uploader tier at http://itmo-app-alb-1330632552.us-east-2.elb.amazonaws.com:8080/
Confirming the endpoint is healthy before we start...
Terminating i-07606d6d741295c02 via the Auto Scaling Group...
Waiting for the endpoint to go down...
Waiting for the endpoint to come back up...
Poll log written to chaos_test_uploader_35292.csv
Recovery time: 82.6s (SLO: 60s)
FAIL: recovery time exceeded the SLO.
```

```
elapsed_seconds,is_up
0.0,True
2.05,False
0.0,False
2.05,False
4.17,False
6.23,False
8.3,False
10.45,False
12.5,False
14.55,False
16.66,False
21.72,False
26.86,False
31.96,False
34.11,False
36.16,False
38.21,False
40.33,False
42.42,False
44.53,False
46.63,False
48.67,False
50.84,False
52.95,False
55.03,False
57.1,False
59.59,False
62.07,False
64.82,False
67.69,False
69.96,False
72.44,False
74.89,False
77.72,False
79.87,False
82.56,True
```

One paragraph explaining, using your own recovery-time numbers, what dominates the recovery window (health-check timing vs. boot/ npm install time vs. something else).

In the chaos test, the instance recovery took 85.18 seconds to get from initial termination to full availability. Analyzing the CSV poll log shows that the EC2 boot time and user-data script take up most of the recovery time. While the Application Load Balancer (ALB) target group detected the original instance failure in just 2.51 seconds, starting up the replacement instance including OS boot, dependency installation (npm install), starting the Node.js application, and passing consecutive target group health checks accounted for the remaining 82.67 seconds.

One paragraph explaining what changed in create_app_stack.sh / delete_app_stack.sh between the previous lab and this one — specifically, what replaced the old run-instances call and why.

In the last lab, create_app_stack.sh used direct aws ec2 run-instances commands to launch standalone EC2 instances directly from launch templates however in this lab, those direct run-instances calls were replaced by Auto Scaling Groups (aws autoscaling create-auto-scaling-group) that were integrated with an Application Load Balancer and target groups. Instead of directly making virtual machines, create_app_stack.sh hands off instance lifecycle management to the ASGs, which automatically maintain the desired capacity across multiple Availability Zones whenever health checks fail or instances are terminated. Correspondingly, delete_app_stack.sh was updated to delete the ASGs (aws autoscaling delete-auto-scaling-group), target groups, and ALB before cleaning up IAM roles and S3 resources.

Screenshot of delete_app_stack.sh completing with no errors

